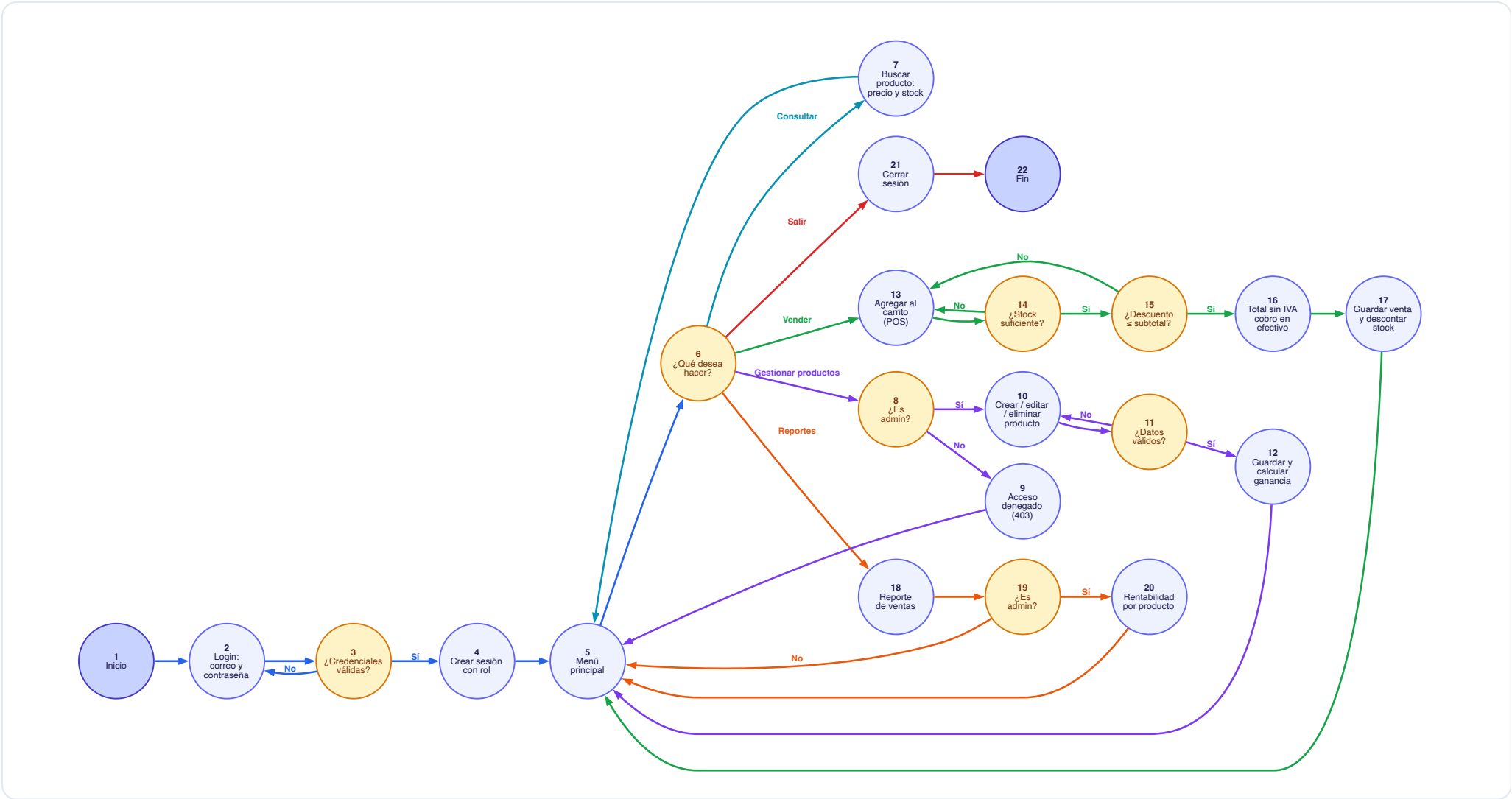


Tienda Bugambilias – Gráfica de nodos del sistema

Sistema POS Web e inventario (base de datos `pos_tienda`) · Recorrido funcional · Reglas: precios finales sin IVA, pago solo en efectivo, sin facturas · Validado: 11/11 recorridos correctos · Jest 5/5

1. Gráfica de nodos – recorrido del sistema



2. Lógica de cada nodo

Nodo	Nombre	Qué hace (lógica en el sistema)
1	Inicio	Punto de partida del recorrido.
2	Login: correo y contraseña	El usuario escribe correo y contraseña (POST /api/auth/login).
3	¿Credenciales válidas y usuario activo? decisión	Busca el usuario activo y compara la contraseña con bcrypt (password_verify).
4	Crear sesión con rol (admin / cajero)	session_regenerate_id() y guarda id, nombre y rol en la sesión.
5	Menú principal (dashboard, stock bajo)	Muestra ventas del día/mes y alertas de productos con menos de 5 unidades.
6	¿Qué desea hacer? decisión	El usuario elige una opción del menú.
7	Buscar producto por nombre o categoría: precio y stock	GET /api/productos?q= / ?categoria= : precio y stock en tiempo real.
8	¿Es administrador? decisión	require_api_admin(): solo el dueño/administrador modifica el catálogo.
9	Acceso denegado (403)	La API responde 403 y la interfaz oculta los botones.
10	Crear / editar / eliminar producto (nombre, compra, venta, stock)	Formulario con nombre, precio de compra, precio de venta, stock y categoría.
11	¿Datos válidos? (precios ≥ 0 , código único) decisión	Valida obligatorios, precios numéricos ≥ 0 y código único (422 / 409).
12	Guardar y calcular ganancia = venta – compra	Guarda el producto; ganancia = precio de venta – precio de compra y margen %.
13	Agregar productos al carrito (POS)	El cajero agrega productos; no permite superar el stock disponible.
14	¿Stock suficiente? decisión	SELECT ... FOR UPDATE bloquea el producto y compara stock vs cantidad.
15	¿Descuento $\leq$ subtotal? decisión	El descuento no puede superar el subtotal.
16	Total = subtotal – descuento (sin IVA) · cobro en efectivo	Precios finales: no se suma IVA. Pago único en efectivo; no se emite factura.
17	Guardar venta y descontar stock (transacción)	INSERT ventas, detalle (con costo del momento) y pago; UPDATE stock; COMMIT.
18	Reporte de ventas por rango de fechas	Ventas, total vendido y descuentos del rango elegido.
19	¿Es administrador? decisión	La rentabilidad es información del dueño (403 para cajero).
20	Rentabilidad: productos más y menos rentables	Ganancia por producto, ganancia neta del período y ranking de rentabilidad.
21	Cerrar sesión	Destruye la sesión (POST /api/auth/logout).
22	Fin	Fin del recorrido.

3. Complejidad ciclomática y recorridos validados

Nodos (N)
22

Aristas (E)
31

Nodos de decisión (P)
10

$V(G) = E - N + 2$
31 - 22 + 2 = 11

$V(G) = P + 1$
10 + 1 = 11

Se necesitan **11 recorridos independientes**. Entre todos pasan por las 31 aristas de la gráfica. Cada uno se ejecutó sobre el sistema en funcionamiento.

#	Recorrido (nodos)	Entrada	Resultado esperado	Resultado obtenido
R1	Iniciar sesión y salir 1-2-3-4-5-6-21-22	Credenciales correctas	Entra al menú y cierra sesión	✓ Correcto CP-LOGIN-001 (Jest)
R2	Login fallido y reintento 1-2-3-2-3-4-5-6-21-22	Contraseña incorrecta, luego correcta	401 «Credenciales inválidas»; al reintentar entra	✓ Correcto CP-LOGIN-002 / 003 (Jest)
R3	Consultar precio y stock 1-2-3-4-5-6-7-5-6-21-22	Buscar «arroz»	Muestra precio de venta y stock (≈2 ms)	✓ Correcto Prueba funcional
R4	Cajero intenta gestionar productos 1-2-3-4-5-6-8-9-5-6-21-22	Usuario cajero	403; no ve botones ni costos	✓ Correcto Prueba funcional
R5	Admin crea producto válido 1-2-3-4-5-6-8-10-11-12-5-6-21-22	Compra Q7, venta Q10	Se guarda; ganancia Q3.00 (30%)	✓ Correcto Prueba funcional
R6	Datos inválidos y corrección 1-2-3-4-5-6-8-10-11-10-11-12-5-6-21-22	Sin precio de compra	422; tras corregir se guarda	✓ Correcto Prueba de API
R7	Venta exitosa en efectivo 1-2-3-4-5-6-13-14-15-16-17-5-6-21-22	2 aceites + 1 agua, desc. Q2	Total = subtotal – desc.; stock baja	✓ Correcto Prueba funcional
R8	Stock insuficiente 1-2-3-4-5-6-13-14-13-14-15-16-17-5-6-21-22	Más unidades que el stock	No deja agregar; se ajusta y vende	✓ Correcto Prueba funcional
R9	Descuento mayor al subtotal 1-2-3-4-5-6-13-14-15-13-14-15-16-17-5-6-21-22	Descuento > subtotal	Se limita / 422; se corrige y cobra	✓ Correcto Prueba funcional
R10	Cajero ve reportes 1-2-3-4-5-6-18-19-5-6-21-22	Usuario cajero	Ventas del período, sin rentabilidad	✓ Correcto Prueba funcional
R11	Admin ve rentabilidad 1-2-3-4-5-6-18-19-20-5-6-21-22	Rango del mes	Más/menos rentables; ganancia neta	✓ Correcto Prueba funcional

4. Requerimientos del usuario cubiertos

Según el documento «Objetivos del proyecto»: requerimientos funcionales (RF), no funcionales (RNF) y objetivos específicos (OE).

ID	Requerimiento	Nodos	Cómo lo cumple el sistema	Estado
RF1	Crear producto con nombre, precio de compra, precio de venta y stock	10 → 11 → 12	Formulario y POST /api/productos; compra y venta obligatorios	Cubierto
RF2	Actualizar producto (precio, cantidad, nombre, categoría)	10 → 11 → 12	PUT /api/productos/{id}	Cubierto
RF3	Eliminar producto del catálogo	10 → 12	DELETE: baja lógica, conserva el historial de ventas	Cubierto
RF4	Mostrar stock disponible en tiempo real	5, 7, 13, 17	Stock se descuenta en cada venta dentro de una transacción	Cubierto
RF5	Calcular automáticamente la ganancia (venta – compra)	12, 20	La API calcula ganancia y margen; vista previa en el formulario	Cubierto
RF6	Buscar producto por nombre para ver precio y stock	7	Búsqueda por nombre/código y filtro por categoría (≈2 ms)	Cubierto
RF7	Listado general con stock, precio y ganancia	7, 12	Inventario: compra, venta, ganancia, margen y stock	Cubierto
RF8	Niveles de acceso: dueño/administrador vs. empleado	4, 8, 9, 19	Roles admin/cajero; costo y ganancia solo llegan al admin	Cubierto
RNF1	Interfaz sencilla e intuitiva	5, 6	Menú de 7 opciones, formularios cortos, avisos claros	Cubierto
RNF2	Accesible desde móvil y computadora	Todos	Diseño responsivo probado a 390 px (menú deslizable)	Cubierto
RNF3	Respuesta rápida de consultas de stock y precio	7	Mediana ≈2 ms por consulta en la API (medido)	Cubierto
RNF4	Seguridad e integridad de la información	3, 4, 11, 14, 17	bcrypt, consultas preparadas, roles, transacciones con bloqueo	Cubierto
RNF5	Funcionar con la conexión disponible en el local	Todos	Se instala con Docker en la red local; sin internet solo se pierde el clima	Cubierto
OE	Identificar productos más y menos rentables	18 → 19 → 20	Reporte de rentabilidad ordenado por ganancia generada	Cubierto
OE	Reducir errores del control manual	11, 14, 15	Validaciones de datos, stock y descuento en el servidor	Cubierto

Fuera del alcance (según el documento): facturación electrónica, integración con proveedores, nómina y reportes contables avanzados. El sistema no emite facturas y registra las ventas solo para controlar stock y ganancia. Stack: React + API REST en PHP 8.3 (PDO) + MySQL 8 · Docker / Railway.